量化交易进阶:第62天 – 从 Alpha 到可交易组合的完整链路（四件套串联）

原创 逻辑与流 逻辑与流 2026年1月30日 18:08 北京 听全文

本文系统讲解从 Alpha 到可交易组合的完整链路,深入剖析机构量化流水线中 Alpha、风险平价、波动率目标、流动性约束、执行成本四件套的串联逻辑与实现。详细阐述 0→1→2→3→4 分层架构的问题出现顺序与输入输出,揭示各层分工与衔接关系。通过完整的 Tushare 多因子回测引擎与 Ablation 五档对比,展示如何逐层开关验证、判读策略死因。帮助读者掌握从「理想化回测」到「接近实盘」的完整流水线,为策略诊断与实盘前压力测试奠定基础。

学习时间

- **理论学习**: 25分钟
- **概念理解**: 20分钟
- **代码实践**: 30分钟
- **总计**: 75分钟

学习目标

你今天要掌握机构量化流水线的完整串联方式：

把一个「可能赚钱的想法」,逐层加工成一个「能长期、稳定、可放大、可执行」的投资产品。

学完你能做到：

- 理解 0→1→2→3→4 各层解决的问题与顺序
- 会在回测中串联 Alpha → RP → VolTarget → Liquidity → Execution
- 会用 Ablation 五档对比定位策略死因
- 能根据流程图与数据流图复现或改造引擎

一句话总结

先在理想世界里找方向（Alpha）,再让风险在资产间公平分摊（Risk Parity）,再把整体风险锁定在目标水平（Vol Target）,再面对现实世界的流动性约束（Liquidity）,最后用真实价格完成交易（Slippage & Impact）。

一、先给你一个「总视角」：机构到底在做什么？

机构做量化，不是在「找一个最牛的公式」，而是在做一件事：

把一个「可能赚钱的想法」， 逐层加工成一个「能长期、稳定、可放大、可执行」的投资产品。

你可以把它理解为一个「标准机构流水线」：

你将看到的 $0 \to 1 \to 2 \to 3 \to 4$， 不是随便分的，而是严格按「问题出现的顺序」分层的。

0) Alpha 层（你已经会）

—— 回答「买什么、相对买多少」（不考虑现实）

- 生成每只股票的 因子得分 score（多因子/ML/GNN 都行）
- 得到「基础选股和偏好」 → 基础权重 $w^{(0)}$
- 例：等权 topN，或按 score 归一化权重

这一步解决：买谁、相对买多少（只回答「偏好」）

这一层只做一件事：

在「假设可以自由交易」的世界里， 判断哪些资产「更值得持有」。

所以它的输入输出是：

- **输入**：价格、财务、情绪、ML 特征……
- **输出**：
  - 因子得分
  - 排名
  - 或一个「基础权重」 $w^{(0)}$

这一步刻意忽略：

- 成本
- 流动性
- 风险
- 规模

为什么要刻意忽略?

因为你要先验证想法是不是有方向性优势（Alpha）。

如果在「理想世界」里都不赚钱， 后面再怎么风控、执行，都是浪费时间。

但 Alpha 层有一个致命问题

它只对「相对好坏」负责，不对结果负责：

- A 比 B 好
- C 比 D 好

但它不关心：

- A 波动是不是巨大?

- A 和 B 是不是同涨同跌?
- 买 A 会不会让整个组合风险爆炸?

这就引出了第 1 层。

1) 风险平价层（Risk Parity）

—— 修正 Alpha 的「风险失真」

把 $w^{(0)}$变成「风险更均匀」的权重 w^{RP}。

- 近似版（工程最常用起步）：$w_i \propto \frac{1}{\sigma_i}$
- 协方差严格版（更专业）：让各资产 RC 相等（需要优化）

这一步解决：不要让高波动/高度相关资产绑架组合风险

1 这一层为什么一定要存在?

因为一个非常现实的事实：

Alpha 强 ≠ 对组合有利

举个直观例子：

- 股票 A：
  - 因子得分高
  - 但波动率 60%，和所有股票高度相关
- 股票 B：
  - 得分一般
  - 波动率 15%，和别人相关性低

如果你直接按 Alpha 权重买：

A 会在组合里 制造绝大部分风险

风险平价层在做什么?

它做的不是「否定 Alpha」，而是：

在不改变「谁更好」这个判断的前提下， 让每只资产「背的风险包」差不多重。

所以你会看到：

- 高波动资产 → 自动被压低权重
- 低波动、分散资产 → 自动被抬高权重

这一层的输出是：

风险更均匀的权重 w^{RP}

为什么风险平价放在 Alpha 后面?

因为逻辑顺序是:

1. 先决定偏好（Alpha）
2. 再校正偏好带来的风险结构

如果你反过来:

- 先做风险平价
- 再叠 Alpha

你会发现 Alpha 的信息被「抹平」。

但风险平价还有一个问题

风险平价只解决了一个问题:

「风险在资产之间怎么分」

它没有解决:

- 整个组合风险是高是低?
- 市场极端波动时怎么办?

这就必须引入第 2 层。

2) 波动率目标层（Vol Target Overlay）

—— 控制「整体风险旋钮」

在得到风险更均匀的 w^{RP}后，再把组合「整体放大或缩小」到目标风险:

$$\sigma_p = \sqrt{w^T \Sigma w} \quad , \quad w^{VT} = w^{RP} \cdot \frac{\sigma_{target}}{\sigma_p}$$

并加上杠杆上限 max_leverage 防止极端放大。

这一步解决：组合总风险恒定（市场波动大就降仓，平稳就加仓）

2 这一层在解决什么问题?

风险平价之后，你的组合可能是:

- 风险结构很均匀 ✔
- 但 整体波动率:
  - 有时 8%
  - 有时 25%
  - 有时 40%

而机构投资产品不能这样。

基金经理必须回答一个问题：

「你这产品的风险水平大概是多少？」

波动率目标层的本质

它引入一个绝对标尺：

我希望这个组合长期年化波动率 ≈ 10%（或 8%、12%）

于是做一件非常关键的事：

- 市场波动上升 → 整体降仓
- 市场平稳 → 整体加仓

这一层不改变：

- 哪些资产重要
- 相对比例

只改变：

- 整体仓位大小

你可以把它理解为：

「风险的总音量旋钮」

为什么它必须在风险平价之后?

如果你先做 Vol Target，再做风险平价：

- 风险平价会再次打乱风险结构
- 你前面的风险控制失效

正确顺序永远是：

先把「内部风险结构」理顺 → 再调「整体风险强度」

到这里还有一个巨大假设

到目前为止，前 0 / 1 / 2 层都在假设一件事：

你算出来的权重，是「可以完整成交的」。

但现实世界马上打你脸。

这就进入第 3 层。

3) 可执行性层：成交量限制（Liquidity Constraint）

—— 从「想买多少」到「能买多少」

你算出来的目标仓位 不一定能成交。

日频常用：

$$Q_{fill} = \min(|Q_{order}|,\ \lambda \cdot ADV)$$

这一步解决：你能不能买到/卖掉那么多（容量与规模）

3 为什么这一层是「生死线」？

因为现实市场里：

不是你想买多少，就能买多少。

尤其当你资金一放大：

- 小市值股票
- 冷门股票
- 高频换手策略

都会遇到一个问题：

市场接不住你。

成交量限制层在干什么？

它把你的目标下单量 Q_{order}， 映射为一个现实可成交量 Q_{fill}。

例如：

- 你想买 50 万股
- 但这只股票 20 日 ADV 只有 300 万
- 你设定参与率 10%

今天最多只能买 30 万股

剩下的：

- 要么不买
- 要么慢慢买（跨日执行）

这一步直接决定：

策略有没有容量（Capacity）

为什么它在 Vol Target 后面？

因为成交量限制关心的是：

- 绝对下单量（股数）

- 而不是权重比例

你必须先把：

- 权重 × NAV → 变成 真实下单量

才能谈「能不能成交」。

即便成交量够，还有一个坑

即便你「能成交」：

成交价 ≠ 你用来算收益的价格

这就是最后一层。

4) 执行成本层：滑点 + 冲击成本（Slippage + Impact）

—— 把「理论价格」变成「真实成交价」

成交价不是 close，而是：

$$P_{exec} = P \cdot \left(1 + sign \cdot \alpha\sqrt{\frac{Q}{ADV}} + \epsilon\right)$$

- $\alpha\sqrt{Q/ADV}$：冲击成本（下单越大越贵）
- ϵ：随机滑点（现实不确定性）

这一步解决：你用什么价格成交（收益不会虚高）

4 这一层在解决什么?

它解决的是：

你「用什么价格成交」

现实里你会遇到：

- 吃深度 → 价格变差
- 对手撤单
- 延迟
- 同时竞争

所以真实成交价一定是：

理论价 ± 成本

为什么它是最后一层?

因为只有在你知道：

- 买多少（成交量限制）
- 在什么时点买

之后，你才能估计：

- 冲击成本
- 随机滑点

这是执行层，不该污染前面的 Alpha / 风险判断。

最终你得到的是

- 可交易、可放大、可控风险 的组合净值

而不是「只在研究里好看」的因子曲线。

二、把 0–4 串成一句「完整逻辑」

现在你可以把整条流水线浓缩成一句话：

先在理想世界里找方向（Alpha），再让风险在资产间公平分摊（Risk Parity），再把整体风险锁定在目标水平（Vol Target），再面对现实世界的流动性约束（Liquidity），最后用真实价格完成交易（Slippage & Impact）。

三、把这套流水线画成一张「结构图 + 数据流图」

A. 结构图（模块分层图）

这张图强调：每一层解决什么问题、输入输出是什么

B. 数据流图（「数据在管道里怎么流」）

这张图强调：你在回测里每个 rebalance 点具体算了什么

记忆诀窍：

- Alpha 决定「方向与偏好」；
- Risk（RP+VolTarget）决定「风险形状与总音量」；
- Liquidity+Execution 决定「现实世界能不能做到、做到要付多少钱」。

四、逐层「关掉」某一层，对比净值：Ablation Study（你亲眼看策略死在哪）

你想做的对比，本质是：同一套 Alpha，只改「现实约束层/风控层」，看看净值如何变化。

我们定义 5 个版本（从理想到现实）：

- **V0**：只开 Alpha（理想世界）
- **V1**：Alpha + 风险平价（RP）
- **V2**：V1 + 波动率目标（VolTarget）
- **V3**：V2 + 成交量限制（Liquidity）
- **V4**：V3 + 冲击/滑点（Execution） ← 最接近真实

你会看到的典型现象（先给你「判读框架」）

- V0 很强，V3/V4 直接崩：策略靠「不可成交/无成本」在赚钱（典型虚高）
- V0→V1 回撤明显下降：你的 alpha 确实被高波动资产绑架过
- V2 比 V1 更稳但收益变慢：Vol Target 把爆仓/大波动期的风险压住了
- V3 之后收益腰斩：容量（参与率/ADV）不支持这个换手/股票池
- V4 再掉一截：冲击/滑点吃掉了边际 alpha（说明 alpha 太薄）

五、实战版：Tushare 多因子回测引擎（四件套已整合，可跑）

特点：

- 用 Tushare 拉取 沪深A股（示例用上证50成分）
- 日频回测（按月调仓，可改）
- 多因子示例（动量、波动率反向、成交量变化）
- 流水线：Alpha → RP → VolTarget → Liquidity → Slippage/Impact → 回测
- 输出：净值曲线、回撤、年化、夏普（简版）+ 关键中间量

你只要换掉 `compute_factors_and_score()` 里的得分，就能接你的真实因子库/ML输出。

（代码见下一节完整版。）

六、你应该怎么用这套引擎（最关键的三点）

1) 你只需要替换一个函数

把你的「因子库/ML输出」接进来，只动这里：

- `compute_factors_and_score(close, volume)`

输出一个 score[date x code] 就行。

2) 四件套每一层都能单独开关验证

你做研究时建议按顺序打开：

1. 只开 Alpha（不加成本/约束）
2. 风险平价（看是否降低回撤）
3. 波动率目标（看是否更稳）
4. 成交量限制（看容量是否崩）
5. 冲击/滑点（看收益是否「还能活」）

这样你能清楚知道：策略死在哪一层。

3) 参数要校准（否则会「误杀策略」）

- IMPACT_ALPHA、RAND_SIGMA、LAMBDA_PART 对结果很敏感
- 真正做实盘前要用历史成交/滑点数据做回归校准
- 研究阶段你可以先用保守参数「压测」

七、代码：在你 Day62 引擎上加「开关」并跑 5 次画一张对比图

下面这段是**「最小侵入式改造」**：你只要把它合并到你那份 day62_engine.py 里即可。

A. 加一个配置开关（建议放在参数区）

```
LAYER_PRESETS = {
    "V0_AlphaOnly":   dict(use_rp=False, use_voltarget=False, use_liquidity=False, use_execu
    "V1_+RiskParity": dict(use_rp=True,  use_voltarget=False, use_liquidity=False, use_execu
    "V2_+VolTarget":  dict(use_rp=True,  use_voltarget=True,  use_liquidity=False, use_execu
    "V3_+Liquidity":  dict(use_rp=True,  use_voltarget=True,  use_liquidity=True,  use_execu
    "V4_+Execution":  dict(use_rp=True,  use_voltarget=True,  use_liquidity=True,  use_execu
}
```

B. 在回测函数里接入开关（核心：4 个 if）

你在 `backtest_day62(...)` 里调仓那段，做如下逻辑替换：

1）权重生成处：RP / VolTarget 开关

```
# picks 已经选出
w0 = np.ones(len(picks)) / len(picks)

# --- (1) Risk Parity: 开关 ---
if flags["use_rp"]:
    w_rp, _ = inv_vol_risk_parity_weights(ret_win)
else:
    w_rp = w0

# --- (2) Vol Target: 开关 ---
```

```
if flags["use_voltarget"]:
    w_final, sig_p, scale = vol_target_overlay(w_rp, cov, SIGMA_TARGET, MAX_LEVERAGE)
else:
    w_final = w_rp
    sig_p, scale = np.nan, 1.0
```

2）下单与成交处：Liquidity / Execution 开关

把你原来计算 q_fill 与 px_exec 的地方改成：

```
q_order_abs = abs(delta_val) / px

# --- (3) Liquidity: 开关 ---
if flags["use_liquidity"]:
    q_fill = cap_fill(q_order_abs, adv_today[c], LAMBDA_PART)
else:
    q_fill = q_order_abs  # 假设100%可成交

if q_fill <= 0:
    continue

# --- (4) Execution: 开关 ---
if flags["use_execution"]:
    px_exec = exec_price(px, side_sign, q_fill, adv_today[c], IMPACT_ALPHA, RAND_SIGMA)
else:
    px_exec = px  # 理想成交价
```

你会发现：

Liquidity 负责「成交多少」，Execution 负责「什么价格成交」。

C. 写一个「跑多次并画对比净值」的入口

在 main() 里把单次回测改成循环：

```
def run_all_versions(close, volume, score):
    nav_dict = {}

    for name, flags in LAYER_PRESETS.items():
        print(f"\n=== Running {name} ===")
        nav_df, lev_df = backtest_day62(close, volume, score, flags=flags)  # 你需要把 flags
        nav_dict[name] = nav_df["nav"]

        nav_df.to_csv(os.path.join(OUTDIR, f"{name}_nav.csv"))

    nav_all = pd.DataFrame(nav_dict).dropna(how="any")
    nav_all.to_csv(os.path.join(OUTDIR, "nav_all_versions.csv"))
```

```
    # 归一化成从1开始，便于对比
    nav_norm = nav_all / nav_all.iloc[0]

    import matplotlib.pyplot as plt
    plt.figure(figsize=(12,5))
    for col in nav_norm.columns:
        plt.plot(nav_norm.index, nav_norm[col], label=col)
    plt.title("Day62 Ablation: 逐层引入现实约束后的净值对比")
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(OUTDIR, "ablation_nav_compare.png"), dpi=150)
    plt.show()

    return nav_norm
```

你需要做的一个小改动是：让 backtest_day62 支持 flags 参数：

```
def backtest_day62(panel_close, panel_volume, score, flags):
    ...
```

你跑完后，我教你「怎么一眼定位策略死因」

把 ablation_nav_compare.png 打开，你按这个顺序问：

1. **V0 vs V1**：RP 是否显著降低回撤/提高夏普?
   - 是：说明之前风险集中很严重
2. **V1 vs V2**：VolTarget 是否在大波动阶段显著抑制回撤?
   - 是：策略在极端行情更可靠
3. **V2 vs V3**：一加 Liquidity 收益是否塌陷?
   - 是：容量不够（换手太高/股票太小/参与率太保守）
4. **V3 vs V4**：一加 Execution 是否再掉一大截?
   - 是：alpha 太薄，被冲击/滑点吃光（或冲击参数过于保守，需要校准）

八、完整代码（四件套 + Ablation 五档）

（以下为完成之后的代码，可直接保存为脚本运行。）

```
import os
import numpy as np
import pandas as pd
import tushare as ts
import matplotlib.pyplot as plt
```

```python
# ========================
# 0) 参数区：你主要改这里
# ========================
INDEX_CODE = "000016.SH"      # 上证50
N_STOCKS = 30                 # 取前N只成分股（先小一点，跑得快）
START = "20180101"            # 拉长到 2018，对齐后有效交易日更多
END   = pd.Timestamp.today().strftime("%Y%m%d")
MIN_TRADING_DAYS = 100        # 有效交易日至少 100 天，否则报错；100~200 会打警告

REBAL_FREQ = "M"              # 调仓频率：'M' 月末 / 'W' 周
TOP_N = 10                    # 选股数量（多头）
LONG_ONLY = True              # True=只做多；False=多空(需要你扩展)

# 风险平价 + vol target
VOL_WINDOW = 20               # 波动/协方差估计窗口（交易日）
SIGMA_TARGET = 0.10           # 目标年化波动率（10%）
MAX_LEVERAGE = 1.0            # 最大总仓位（1.0=不加杠杆；>1允许杠杆）

# 流动性约束
ADV_WINDOW = 20               # ADV窗口
LAMBDA_PART = 0.10            # 最大参与率（10%） -> 最大成交量 = 10% * ADV

# 交易成本
COMMISSION = 0.0003           # 单边手续费（万三）
IMPACT_ALPHA = 0.10           # 冲击系数（经验参数，需校准）
RAND_SIGMA = 0.02             # 随机滑点强度（配合Q/ADV缩放）

INITIAL_CASH = 1_000_000
TRADING_DAYS = 252

# 四件套逐层开关（做 ablation：逐层打开验证）
LAYER_PRESETS = {
    "V0_AlphaOnly":   dict(use_rp=False, use_voltarget=False, use_liquidity=False, use_execu
    "V1_+RiskParity": dict(use_rp=True,  use_voltarget=False, use_liquidity=False, use_execu
    "V2_+VolTarget":  dict(use_rp=True,  use_voltarget=True,  use_liquidity=False, use_execu
    "V3_+Liquidity":  dict(use_rp=True,  use_voltarget=True,  use_liquidity=True,  use_execu
    "V4_+Execution":  dict(use_rp=True,  use_voltarget=True,  use_liquidity=True,  use_execu
}

OUTDIR = "day62_output"
os.makedirs(OUTDIR, exist_ok=True)

# ========================
# 1) Tushare 初始化
# ========================
TOKEN = "3f21bb0ff0889eb10b050c7536ca4a55064af3397f22e43d74b7ccb0"
if not TOKEN:
    raise ValueError("请先设置环境变量：export TUSHARE_TOKEN='你的token'")
ts.set_token(TOKEN)
```

```python
pro = ts.pro_api()

plt.rcParams["font.sans-serif"] = ["SimHei", "Arial Unicode MS", "Noto Sans CJK SC", "DejaVu
plt.rcParams["axes.unicode_minus"] = False


# ======================
# 2) 数据：取成分股 & 拉日线
# ======================
def _to_ts_code(s):
    """con_code 转为 pro.daily 需要的 ts_code（补全 .SH/.SZ）"""
    s = str(s).strip()
    if s.endswith(".0") and s[:-2].isdigit():
        s = s[:-2]
    if "." in s and (s.endswith(".SH") or s.endswith(".SZ")):
        return s
    if len(s) < 6:
        return s
    return s + (".SH" if s[0] in ("6", "9") else ".SZ")


def get_index_constituents(index_code, n, trade_date):
    wdf = pro.index_weight(index_code=index_code, trade_date=trade_date)
    if wdf.empty:
        fallback = (pd.to_datetime(trade_date) - pd.Timedelta(days=7)).strftime("%Y%m%d")
        wdf = pro.index_weight(index_code=index_code, trade_date=fallback)
    if wdf.empty:
        raw = [
            "600519.SH", "601318.SH", "600036.SH", "600030.SH", "601166.SH",
            "000858.SZ", "000333.SZ", "601288.SH", "601398.SH", "600276.SH",
            "600887.SH", "601012.SH", "601857.SH", "600585.SH", "600309.SH",
            "601899.SH", "600050.SH", "601628.SH", "000568.SZ", "601818.SH",
        ]
        return [_to_ts_code(c) for c in raw[:n]]
    wdf = wdf.sort_values("weight", ascending=False)
    col = "con_code" if "con_code" in wdf.columns else "code"
    raw = wdf[col].dropna().astype(str).head(n * 2).tolist()
    return [_to_ts_code(c) for c in raw[:n]]

def fetch_daily_bars(ts_code, start, end):
    df = pro.daily(ts_code=ts_code, start_date=start, end_date=end,
                   fields="ts_code,trade_date,open,high,low,close,vol")
    if df.empty:
        return df
    df = df.sort_values("trade_date")
    df["date"] = pd.to_datetime(df["trade_date"])
    df = df.set_index("date")
    return df[["open","high","low","close","vol"]].rename(columns={"vol":"volume"})
```

```python
# ========================
# 3) 因子与得分（你可替换成你的因子库/ML输出）
# ========================
def compute_factors_and_score(panel_close, panel_volume):
    ret = panel_close.pct_change()
    mom20 = panel_close / panel_close.shift(20) - 1.0
    vol20 = ret.rolling(20).std() * np.sqrt(TRADING_DAYS)
    vchg5 = panel_volume.pct_change(5)

    def zscore_cs(df):
        m = df.mean(axis=1)
        s = df.std(axis=1).replace(0, np.nan)
        return df.sub(m, axis=0).div(s, axis=0)

    mom_z = zscore_cs(mom20)
    vol_z = zscore_cs(vol20)
    vchg_z = zscore_cs(vchg5)
    score = 0.6 * mom_z + (-0.3) * vol_z + 0.1 * vchg_z
    return score


# ========================
# 4) 风险平价（近似版：波动率倒数）
# ========================
def inv_vol_risk_parity_weights(returns_window: pd.DataFrame):
    vol = returns_window.std() * np.sqrt(TRADING_DAYS)
    inv = 1.0 / np.maximum(vol.values, 1e-12)
    w = inv / inv.sum()
    return w, vol.values

def portfolio_sigma(weights, cov):
    w = np.asarray(weights).reshape(-1,1)
    return float(np.sqrt(w.T @ cov @ w))

def vol_target_overlay(weights, cov, sigma_target, max_leverage):
    sig = portfolio_sigma(weights, cov)
    scale = (sigma_target / sig) if sig > 1e-12 else 0.0
    scale = min(scale, max_leverage)
    return weights * scale, sig, scale


# ========================
# 5) 流动性约束：ADV 参与率限制（部分成交）
# ========================
def calc_adv(vol_series: pd.Series, window=20):
    return vol_series.rolling(window, min_periods=1).mean()

def cap_fill(q_order_abs, adv, lambda_part):
    max_fill = max(0.0, lambda_part * float(adv))
    q_fill = min(float(q_order_abs), max_fill)
    return q_fill
```

```python
# ========================
# 6) 执行价：冲击成本 + 随机滑点
# ========================
def exec_price(base_price, side_sign, qty_abs, adv, alpha, rand_sigma):
    adv = max(float(adv), 1e-12)
    x = float(qty_abs) / adv
    impact = alpha * np.sqrt(max(x, 0.0))
    eps = np.random.normal(0.0, rand_sigma * x)
    return base_price * (1.0 + side_sign * impact + eps)


# ========================
# 7) 回测主循环（事件驱动：调仓日交易）
# ========================
def backtest_day62(panel_close, panel_volume, score, flags=None):
    if flags is None:
        flags = dict(use_rp=True, use_voltarget=True, use_liquidity=True, use_execution=True
    codes = list(panel_close.columns)
    dates = panel_close.index

    ret = panel_close.pct_change()
    adv = panel_volume.apply(lambda s: calc_adv(s, ADV_WINDOW))
    rebalance_dates = panel_close.resample(REBAL_FREQ).last().index.intersection(dates)

    cash = INITIAL_CASH
    pos_shares = pd.Series(0.0, index=codes)
    nav_list = []
    leverage_list = []

    for dt in dates:
        price_today = panel_close.loc[dt]
        vol_today = panel_volume.loc[dt]
        adv_today = adv.loc[dt]

        holdings_value = float((pos_shares * price_today).sum())
        nav = cash + holdings_value
        nav_list.append((dt, nav))

        if dt in rebalance_dates:
            if ret.loc[:dt].shape[0] < VOL_WINDOW + 5:
                continue

            sc = score.loc[dt].dropna()
            if sc.empty:
                continue
            picks = sc.sort_values(ascending=False).head(TOP_N).index.tolist()

            w0 = np.ones(len(picks)) / len(picks)
            ret_win = ret[picks].loc[:dt].tail(VOL_WINDOW).dropna(how="any")
            if ret_win.shape[0] < max(10, VOL_WINDOW // 2):
                continue
```

```
            cov = ret_win.cov().values * TRADING_DAYS

            if flags["use_rp"]:
                w_rp, _ = inv_vol_risk_parity_weights(ret_win)
            else:
                w_rp = w0.copy()

            if flags["use_voltarget"]:
                w_final, sig_p, scale = vol_target_overlay(w_rp, cov, SIGMA_TARGET, MAX_LEVE
            else:
                w_final = w_rp.copy()
                scale = 1.0
            leverage_list.append((dt, float(w_final.sum())))

            target_value = pd.Series(0.0, index=codes)
            for i, c in enumerate(picks):
                target_value[c] = nav * w_final[i]

            for c in codes:
                px = float(price_today[c])
                if not np.isfinite(px) or px <= 0:
                    continue

                cur_sh = float(pos_shares[c])
                cur_val = cur_sh * px
                tgt_val = float(target_value[c])
                delta_val = tgt_val - cur_val
                if abs(delta_val) < 1e-6:
                    continue

                side_sign = 1.0 if delta_val > 0 else -1.0
                q_order_abs = abs(delta_val) / px

                if flags["use_liquidity"]:
                    q_fill = cap_fill(q_order_abs, adv_today[c], LAMBDA_PART)
                else:
                    q_fill = q_order_abs
                if q_fill <= 0:
                    continue

                if flags["use_execution"]:
                    px_exec = exec_price(px, side_sign, q_fill, adv_today[c], IMPACT_ALPHA,
                else:
                    px_exec = px

                trade_val = q_fill * px_exec
                fee = trade_val * COMMISSION

                if side_sign > 0:
                    total_cost = trade_val + fee
                    if total_cost > cash:
                        ratio = cash / max(total_cost, 1e-12)
                        q_fill2 = q_fill * ratio
                        if q_fill2 <= 0:
```

```
                            continue
                        if flags["use_execution"]:
                            px_exec2 = exec_price(px, side_sign, q_fill2, adv_today[c], IMPA
                        else:
                            px_exec2 = px
                        trade_val2 = q_fill2 * px_exec2
                        fee2 = trade_val2 * COMMISSION
                        cash -= (trade_val2 + fee2)
                        pos_shares[c] += q_fill2
                    else:
                        cash -= total_cost
                        pos_shares[c] += q_fill
                else:
                    sell_qty = min(q_fill, cur_sh)
                    if sell_qty <= 0:
                        continue
                    trade_val = sell_qty * px_exec
                    fee = trade_val * COMMISSION
                    cash += (trade_val - fee)
                    pos_shares[c] -= sell_qty

    nav_df = pd.DataFrame(nav_list, columns=["date","nav"]).set_index("date")
    nav_df["ret"] = nav_df["nav"].pct_change().fillna(0.0)
    return nav_df, pd.DataFrame(leverage_list, columns=["date","gross_exposure"]).set_index(


# ======================
# 8) 绩效指标（简版）
# ======================
def perf_stats(nav_df):
    r = nav_df["ret"]
    ann_ret = (nav_df["nav"].iloc[-1] / nav_df["nav"].iloc[0]) ** (TRADING_DAYS / len(nav_df
    ann_vol = r.std() * np.sqrt(TRADING_DAYS)
    sharpe = ann_ret / max(ann_vol, 1e-12)
    cum = (1 + r).cumprod()
    peak = cum.cummax()
    dd = cum / peak - 1
    mdd = dd.min()
    return {
        "ann_return": ann_ret,
        "ann_vol": ann_vol,
        "sharpe": sharpe,
        "max_drawdown": mdd
    }


# ======================
# 9) Ablation: 跑 5 档预设并画对比图
# ======================
def run_all_versions(close, volume, score):
    nav_dict = {}
    stats_list = []
```

```
    for name, fl in LAYER_PRESETS.items():
        print("\n=== Running %s ===" % name)
        nav_df, lev_df = backtest_day62(close, volume, score, flags=fl)
        nav_dict[name] = nav_df["nav"]
        nav_df.to_csv(os.path.join(OUTDIR, "%s_nav.csv" % name))
        st = perf_stats(nav_df)
        stats_list.append((name, st))
        print("  ann_return=%.4f  sharpe=%.4f  max_drawdown=%.4f" % (st["ann_return"], st["s

    nav_all = pd.DataFrame(nav_dict).dropna(how="any")
    nav_all.to_csv(os.path.join(OUTDIR, "nav_all_versions.csv"))
    nav_norm = nav_all / nav_all.iloc[0]

    plt.figure(figsize=(12, 5))
    for col in nav_norm.columns:
        plt.plot(nav_norm.index, nav_norm[col], label=col)
    plt.title("Day62 Ablation：逐层引入现实约束后的净值对比")
    plt.xlabel("日期")
    plt.ylabel("归一化净值（起点=1）")
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(OUTDIR, "ablation_nav_compare.png"), dpi=150)
    plt.show()

    print("\n===== Ablation 汇总（便于逐点验证）=====")
    for name, st in stats_list:
        print("%s: ann_return=%.4f  sharpe=%.4f  max_drawdown=%.4f" % (name, st["ann_return"
    return nav_norm


# ======================
# 10）主程序
# ======================
def main():
    trade_date = END
    codes = get_index_constituents(INDEX_CODE, N_STOCKS, trade_date)
    print("股票池：", codes)
    bars = {}
    for c in codes:
        df = fetch_daily_bars(c, START, END)
        if df.empty:
            continue
        bars[c] = df

    if len(bars) == 0:
        raise ValueError(
            "所有成分股日线均无数据（pro.daily 返回空）。请检查：1）Tushare 积分/权限；"
            "2）日期 %s~%s；3）成分股代码格式需带 .SH/.SZ（如 600519.SH）。" % (START, END)
        )
```

```
    close = pd.DataFrame({c: bars[c]["close"] for c in bars}).sort_index()
    volume = pd.DataFrame({c: bars[c]["volume"] for c in bars}).sort_index()
    close = close.dropna(how="any")
    volume = volume.loc[close.index].dropna(how="any")

    n_days = close.shape[0]
    print("成功拉取 %d 只，对齐后有效交易日数：%d" % (len(bars), n_days))
    if n_days < MIN_TRADING_DAYS:
        if n_days == 0:
            print("各股日期范围示例：")
            for i, c in enumerate(list(bars.keys())[:5]):
                ix = bars[c].index
                print("  %s: %s ~ %s" % (c, ix.min().strftime("%Y-%m-%d"), ix.max().strftime
        raise ValueError(
            "有效交易日太少（%d < %d）。请扩大日期范围（START 更早）或减少 N_STOCKS。"
            % (n_days, MIN_TRADING_DAYS)
        )
    if n_days < 200:
        print("注意：有效交易日 %d 略少，回测区间偏短，建议将 START 提前或减少成分股数以提高对齐天数。" %

    score = compute_factors_and_score(close, volume)
    run_all_versions(close, volume, score)

    print("\n结果已保存到 %s：" % OUTDIR)
    print("  - V0_AlphaOnly_nav.csv ~ V4_+Execution_nav.csv：各档净值")
    print("  - nav_all_versions.csv：五档对齐后的净值表")
    print("  - ablation_nav_compare.png：逐层引入约束后的净值对比图")

if __name__ == "__main__":
    main()
```

九、这张图在说什么？

图名：Day62 Ablation：逐层引入现实约束后的净值对比

- **横轴**：时间（回测区间内的每个交易日）
- **纵轴**：归一化净值（起点都设为 1，方便比较谁涨得多、谁跌得少）
- **每条曲线**：一种「配置」——从只做 Alpha（V0）到四件套全开（V4），共 5 条。

五条曲线分别代表什么？

曲线	含义
V0_AlphaOnly	纯 Alpha：只选股、等权、不控风险、不限制成交量、不考虑冲击/滑点（理想化基准）
V1_+RiskParity	在 V0 基础上 + 风险平价（RP）：权重按风险贡献调，降低单一资产主导

曲线	含义
V2_+VolTarget	在 V1 基础上 + 波动率目标：整体仓位按目标波动率缩放，大波动时自动降仓
V3_+Liquidity	在 V2 基础上 + 流动性约束：单日成交量不能超过 ADV×参与率，模拟「不能一口气买太多」
V4_+Execution	在 V3 基础上 + 执行层：成交价考虑冲击成本和随机滑点（最接近实盘）

这张图能说明什么？

1. **策略「死」在哪一层**
   看从哪一条开始，曲线明显比上一条差一大截（收益掉下去、回撤变大）。那一层就是当前策略的「瓶颈」：要么在这一层上改（调参数/换约束），要么承认容量或执行成本把 Alpha 吃掉了。

2. **每层约束的代价**

   - V0→V1：加 RP 后，曲线若更平滑、回撤更小 → 说明之前风险太集中，RP 有用。
   - V1→V2：加 Vol Target 后，若在大波动段明显少跌 → 说明控波动有效，策略更稳。
   - V2→V3：一加 Liquidity，若收益明显塌下去 → 说明容量/换手/参与率有问题（要么股票太小，要么参与率设太严）。
   - V3→V4：一加 Execution，若再掉一截 → 说明 Alpha 偏薄，被冲击/滑点吃掉了，或冲击参数设得偏保守（需要校准）。

3. **实盘预期**
   V4 最接近真实交易（有 RP、Vol Target、流动性限制、冲击滑点）。若 V4 曲线仍然明显高于 1、且回撤可接受，说明策略在「加完现实约束后」还有空间；若 V4 已经很平甚至往下，说明当前 Alpha 在实盘环境下可能不够用。

4. **研究顺序**
   图的设计就是「按顺序一层层加约束」：先看纯 Alpha（V0），再看加风险控制（V1、V2），再看容量（V3），最后看执行成本（V4）。这样你可以清楚知道：是风险控制、流动性还是执行成本在「杀」你的策略。

一句话总结

这张图就是在回答：从「理想化回测」到「接近实盘」的过程中，每加一层现实约束，收益和风险会变成什么样；以及你的策略主要是在哪一层被「打掉」的。用来做策略诊断和实盘前的压力测试最合适。

结论

先在理想世界里找方向（Alpha），再让风险在资产间公平分摊（Risk Parity），再把整体风险锁定在目标水平（Vol Target），再面对现实世界的流动性约束（Liquidity），最后用真实价格完成交易（Slippage & Impact）。

记忆诀窍：Alpha 决定「方向与偏好」；Risk（RP+VolTarget）决定「风险形状与总音量」；Liquidity+Execution 决定「现实世界能不能做到、做到要付多少钱」。从「理想化回测」到「接近实盘」，每加一层现实约束都会改变收益与风险；用 Ablation 五档对比可以一眼定位策略死因，是做策略诊断和实盘前压力测试的合适工具。

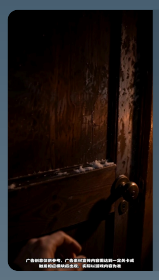
量化交易入门指南 · 目录

<上一篇
Day 61 | 风险平价仓位分配——各资产按波动率平衡风险（Risk Parity）

下一篇 >
量化交易进阶:第63天 – 动态再平衡策略（触发式/定期调仓机制）（进阶阶段·回测框架篇）

阅读 92

广告



寒冬没有退路，合理分配每份资源才能守住营地。

点击了解更多 >

寒冬没有退路，合理分配每份资源才能守住营地。

无尽冬日–3亿人都在玩

立即体验